

JAVA DIGITAL PAYMENT SYSTEM

Object-Oriented Programming Practical Record

Question - Sample Case Study on Interfaces, Classes & Custom Exceptions (CO1)

Digital Payment / UPI Wallet Management System

Problem Statement: Design a system to manage a digital wallet and process UPI payments.

Classes: Wallet, UPIPayment.

Interface: PaymentService.

Exceptions: InvalidUPIException, InvalidAmountException, InsufficientBalanceException.

Attributes: User name, mobile number, UPI ID, balance.

Methods: Add Money, Pay, Check Balance, Display Wallet Details, etc.

Object Usage: Create a wallet object and perform operations such as adding money, checking balance, and making UPI payments.

Aim: To implement this menu-driven system in Java using interfaces, encapsulation, String validation, custom exceptions, and exception handling.

Submitted by: G. Pranadeep

Roll No.: A25126552212

Class: 2/4 CSM-D

Branch: Computer Science (Artificial Intelligence / Machine Learning)

Java Digital Payment System - Practical Record

1. Steps for Execution

1. Open any Java development environment such as VS Code, IntelliJ IDEA, Eclipse, or a command prompt with the JDK installed.
2. Create a new Java source file named DigitalPaymentSystem.java.
3. Copy the complete program from this record into the file and save it.
4. Compile the source file using `javac DigitalPaymentSystem.java`.
5. Run the compiled program using `java DigitalPaymentSystem`.
6. Choose an option from the displayed menu, enter the requested wallet/amount/UPI values, and continue until option 6 is selected.

Initial Wallet Data

Field	Example Value
User Name	Rahul
Mobile Number	9876543210
UPI ID	rahul@upi
Opening Balance	Rs.0

2. Algorithm

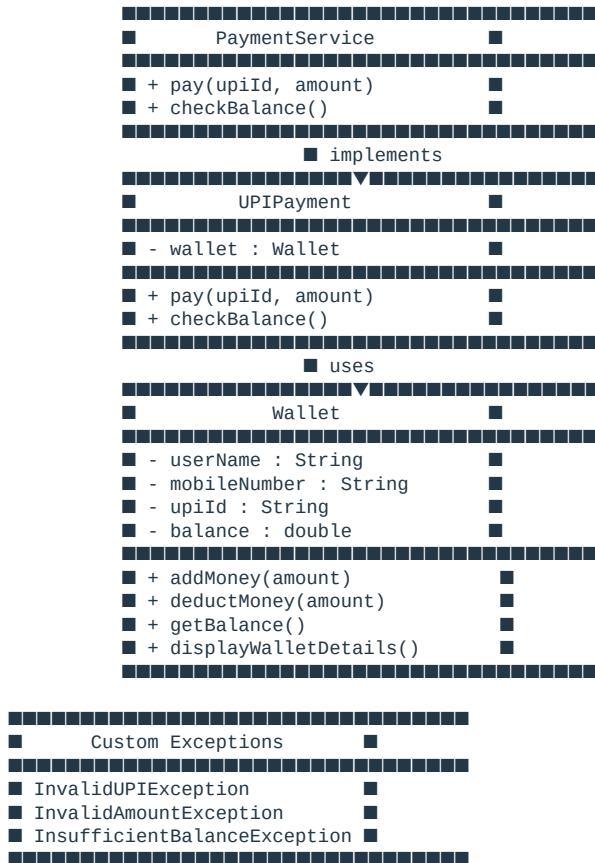
1. Start the program and create a Scanner object to receive keyboard input.
2. Display the digital payment menu and read the user's choice.
3. For Create Wallet, read the user's name, mobile number, and UPI ID and create a Wallet object.
4. For Add Money, read the amount and add it only when the amount is positive.
5. For Make UPI Payment, read the receiver UPI ID and payment amount.
6. Validate the receiver UPI ID using String operations. An invalid format causes InvalidUPIException.
7. Reject zero or negative transaction amounts using InvalidAmountException.
8. Check the wallet balance. If the amount is greater than the balance, throw InsufficientBalanceException.
9. For a valid payment, deduct the amount and display the payment success message.
10. Use try-catch-finally to handle exceptions and display transaction status.
11. Repeat the menu until option 6 is selected, then display final wallet details and stop.

Java Digital Payment System - Practical Record

3. UML Class Diagram

The diagram shows the PaymentService interface, the UPIPayment implementation, and the Wallet class used for wallet operations.

UML CLASS DIAGRAM



Key OOP Concepts: interface implementation (UPIPayment implements PaymentService), encapsulation (private Wallet data members), object association (UPIPayment uses Wallet), String validation, and exception handling through custom exceptions.

Relationship: UPIPayment implements PaymentService and uses a Wallet object for payment and balance operations.

Java Digital Payment System - Practical Record

4. Java Source Code

Filename: DigitalPaymentSystem.java

```
import java.util.Scanner;

interface PaymentService {
    void pay(String upiId, double amount) throws InvalidUPIException, InvalidAmountException, InsufficientBalanceException;
    void checkBalance();
}

class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) { super(message); }
}

class InvalidUPIException extends Exception {
    public InvalidUPIException(String message) { super(message); }
}

class InvalidAmountException extends Exception {
    public InvalidAmountException(String message) { super(message); }
}

class Wallet {
    private String userName;
    private String mobileNumber;
    private String upiId;
    private double balance;

    public Wallet(String userName, String mobileNumber, String upiId) {
        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upiId = upiId;
        this.balance = 0;
    }

    public void addMoney(double amount) throws InvalidAmountException {
        if (amount <= 0) throw new InvalidAmountException("Amount must be greater than zero.");
        balance += amount;
        System.out.println("Rs." + amount + " added successfully.");
    }

    public void deductMoney(double amount) { balance -= amount; }
    public double getBalance() { return balance; }

    public void displayWalletDetails() {
        System.out.println("\n----- Wallet Details -----");
        System.out.println("User Name      : " + userName);
        System.out.println("Mobile Number : " + mobileNumber);
        System.out.println("UPI ID       : " + upiId);
        System.out.println("Balance      : Rs." + balance);
    }
}
```

Java Digital Payment System - Practical Record

4. Java Source Code (continued)

```
class UPIPayment implements PaymentService {
    private Wallet wallet;

    public UPIPayment(Wallet wallet) { this.wallet = wallet; }

    @Override
    public void pay(String upiId, double amount) throws InvalidUPIException, InvalidAmountException, InsufficientBalanceException {
        if (upiId == null || !upiId.contains("@") || upiId.startsWith("@") || upiId.endsWith("@") || upiId.indexOf("@") != upiId.lastIndexOf("@"))
            throw new InvalidUPIException("Invalid UPI ID.");
        if (amount <= 0) throw new InvalidAmountException("Payment amount must be greater than zero.");
        if (amount > wallet.getBalance())
            throw new InsufficientBalanceException("Insufficient balance. Available balance: Rs." + wallet.getBalance());

        wallet.deductMoney(amount);
        System.out.println("\n===== PAYMENT SUCCESS =====");
        System.out.println("Amount Paid : Rs." + amount);
        System.out.println("Paid To      : " + upiId);
        System.out.println("Remaining    : Rs." + wallet.getBalance());
        System.out.println("=====");
    }

    @Override
    public void checkBalance() { System.out.println("Available Balance: Rs." + wallet.getBalance()); }
}

public class DigitalPaymentSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Wallet wallet = null;
        PaymentService payment = null;
        int choice;

        do {
            System.out.println("\n===== DIGITAL PAYMENT SYSTEM =====");
            System.out.println("1. Create Wallet");
            System.out.println("2. Add Money");
            System.out.println("3. Make UPI Payment");
            System.out.println("4. Check Balance");
            System.out.println("5. Display Wallet Details");
            System.out.println("6. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();
            sc.nextLine();

            try {
                switch (choice) {
                    case 1:
                        if (wallet != null) { System.out.println("Wallet already exists!"); break; }
                        System.out.print("Enter your name: ");
                        String name = sc.nextLine();
                        System.out.print("Enter mobile number: ");
                        String mobile = sc.nextLine();
                        System.out.print("Enter your UPI ID: ");
                        String upi = sc.nextLine();
                        wallet = new Wallet(name, mobile, upi);
                        payment = new UPIPayment(wallet);
                        System.out.println("Wallet created successfully!");
                        break;
                    case 2:
                        if (wallet == null) { System.out.println("Please create a wallet first."); break; }
                        System.out.print("Enter amount to add: Rs.");
                        double addAmount = sc.nextDouble();
                        wallet.addMoney(addAmount);
                        System.out.println("Current Balance: Rs." + wallet.getBalance());
                        break;
                    case 3:
                        if (wallet == null) { System.out.println("Please create a wallet first."); break; }
                        System.out.print("Enter receiver UPI ID: ");
                        String receiverUPI = sc.nextLine();
                        System.out.print("Enter payment amount: Rs.");
                        double paymentAmount = sc.nextDouble();
                        payment.pay(receiverUPI, paymentAmount);
                        break;
                }
            }
        }
    }
}
```

Java Digital Payment System - Practical Record

4. Java Source Code (continued)

```
        case 4:
            if (wallet == null) { System.out.println("Please create a wallet first."); break; }
            payment.checkBalance();
            break;
        case 5:
            if (wallet == null) { System.out.println("Please create a wallet first."); break; }
            wallet.displayWalletDetails();
            break;
        case 6:
            System.out.println("Thank you for using Digital Payment System!");
            break;
        default:
            System.out.println("Invalid choice. Please try again.");
    }
} catch (InvalidUPIException e) {
    System.out.println("Transaction Failed: " + e.getMessage());
} catch (InvalidAmountException e) {
    System.out.println("Transaction Failed: " + e.getMessage());
} catch (InsufficientBalanceException e) {
    System.out.println("Transaction Failed: " + e.getMessage());
} finally {
    System.out.println("Transaction processing completed.");
}
} while (choice != 6);

if (wallet != null) {
    System.out.println("\nFINAL WALLET DETAILS");
    wallet.displayWalletDetails();
}
sc.close();
}
```

The source code is kept compact with minimal unnecessary blank lines, while retaining the required interface, wallet class, payment implementation, menu, and custom exception handling.

Compilation & Execution Commands

Save the source code exactly as:

DigitalPaymentSystem.java

Compile:

javac DigitalPaymentSystem.java

Run:

java DigitalPaymentSystem

Java Digital Payment System - Practical Record

5. Representative Sample Output

The run below uses these inputs in sequence: create a wallet for Rahul; add Rs.5,000; make a UPI payment of Rs.1,000 to priya@upi; display the final wallet details; exit.

```
===== DIGITAL PAYMENT SYSTEM =====
1. Create Wallet
2. Add Money
3. Make UPI Payment
4. Check Balance
5. Display Wallet Details
6. Exit
Enter your choice: 1
Enter your name: Rahul
Enter mobile number: 9876543210
Enter your UPI ID: rahul@upi
Wallet created successfully!
Transaction processing completed.

Enter your choice: 2
Enter amount to add: Rs.5000
Rs.5000.0 added successfully.
Current Balance: Rs.5000.0
Transaction processing completed.

Enter your choice: 3
Enter receiver UPI ID: priya@upi
Enter payment amount: Rs.1000

===== PAYMENT SUCCESS =====
Amount Paid : Rs.1000.0
Paid To      : priya@upi
Remaining    : Rs.4000.0
=====
Transaction processing completed.

Enter your choice: 5

----- Wallet Details -----
User Name    : Rahul
Mobile Number : 9876543210
UPI ID       : rahul@upi
Balance      : Rs.4000.0

Enter your choice: 6
Thank you for using Digital Payment System!
Transaction processing completed.

FINAL WALLET DETAILS
----- Wallet Details -----
User Name    : Rahul
Mobile Number : 9876543210
UPI ID       : rahul@upi
Balance      : Rs.4000.0
```

Result

The Digital Payment System was successfully implemented in Java using an interface, classes, objects, encapsulation, String validation, and custom exceptions. The program supports wallet creation, adding money, UPI payment processing, balance checking, invalid UPI detection, invalid amount detection, insufficient balance handling, and final wallet details display.